

README

HXC-029 — full-qa-api Bank: Self-Driving + Verified vs Live Server

Field	Value
Revision	1
Created	2026-05-29
Last modified	2026-05-29
Status	active
Tracked item	HXC-029 (§11.4.98 full-automation anti-bluff forward sweep)
Pass type	Real runtime execution against the LIVE HelixCode server (real PostgreSQL + Redis backed). Captured evidence below.

Table of contents

- [What this is](#)
- [Critical finding: the original bank targets a different product](#)
- [What was produced](#)
- [Runner invocation](#)
- [Three-run results \(§11.4.98 re-runnability\)](#)
- [Anti-bluff mutation proof](#)
- [Real endpoints exercised](#)
- [Honest skips](#)
- [Evidence files](#)
- [Repeatability for the other banks](#)

What this is

The HXC-029 deliverable: take ONE HelixQA bank and make it genuinely self-driving and §11.4.98-compliant against the live HelixCode server on `http://localhost:8080`, with captured evidence, establishing the rewrite pattern for the remaining banks.

Critical finding: the original bank targets a different product

`helix_qa/banks/full-qa-api.json` declares `metadata.app = "Catalogizer"` and was machine-converted (`converted-by-bank-prose-to-http.py`) for the **Catalogizer catalog-api**. Its 331 `http:` steps hit endpoints such as `/api/v1/entities`, `/api/v1/collections`, `/api/v1/scans` and expect a `session_token` login with `admin/admin123`. Probed live against HelixCode:

- `POST /api/v1/auth/login` with `admin/admin123` → **401** (no such user)

- GET /api/v1/entities → 404 (route does not exist on HelixCode)

So the Catalogizer bank cannot honestly run against HelixCode — rewriting its 27 manual-review-required prose steps in place would leave 331 catalog-api http: steps failing en masse, which is not a clean §11.4.98 demonstration. The original bank is left untouched (it is valid for its own product).

Instead, a **new HelixCode-targeted API bank** was authored against HelixCode's real route table (helix_code/internal/server/server.go), discovered by live probing. This is the correct §11.4.98 artefact for the running server.

What was produced

- helix_qa/banks/helixcode-full-qa-api.json — 30 test cases / 34 steps: 32 self-driving http:+assert steps against real HelixCode endpoints, 2 honest_skip steps (JWT-mint, WebSocket) with reasons. No manual-review-required prose, no human action after startup, read-only / negative paths only (self-cleaning — no state to clean up).
- runner/main.go — a stdlib-only standalone runner that implements the documented HelixQA http:/assert executor contract (see helix_qa/pkg/testbank/schema.go + helix_qa/pkg/autonomous/http_executor.go) and drives the live server. Needed because helix_qa/pkg/autonomous does not compile in this checkout (its module replace directives point at owned submodules — DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine — that are not checked out; a topology issue unrelated to the HTTP bank). The runner makes REAL HTTP calls and REAL assertions — it is not a mock.

Runner invocation

```
cd docs/qa/HXC-029/full-qa-api/runner
go build -o /tmp/hxc029runner .
HELIXQA_HTTP_BASE_URL=http://localhost:8080 \
/tmp/hxc029runner -bank ../../../../helix_qa/banks/
helixcode-full-qa-api.json
```

(The same bank JSON is consumed by the real helix_qa HTTPExecutor / TestBankRealBinary_FullQAAPI path once the missing submodules are checked out — the action schema is identical.)

Three-run results (§11.4.98 re-runnability)

run_1.txt, run_2.txt, run_3.txt — three consecutive automated invocations, no human action between or during:

```
=== summary: 32 PASS, 0 FAIL, 2 SKIP (of 34 steps) === (run 1, exit 0)
=== summary: 32 PASS, 0 FAIL, 2 SKIP (of 34 steps) === (run 2, exit 0)
=== summary: 32 PASS, 0 FAIL, 2 SKIP (of 34 steps) === (run 3, exit 0)
```

Deterministic and identical across runs (the -count=3 equivalent). Self-cleaning: every step is read-only or a negative path, so state is identical at the start of each run.

Anti-bluff mutation proof

mutation-proof.txt: a copy of the bank with HXC-API-001's expect_status changed 200 → 999 was run against the same live server. The runner reported **FAIL** with exit code 1:

```
FAIL [HXC-API-001] GET /health — GET /health -> status 200, expected 999 .
=== summary: 31 PASS, 1 FAIL, 2 SKIP (of 34 steps) ===
```

This proves the PASSes are real assertions against the live server — when the expectation disagrees with reality, the run fails.

Real endpoints exercised

endpoint-probes.txt captures the raw curl probes proving each endpoint is real. The bank asserts on:

- GET /health, GET /api/v1/health → 200, status=healthy, version
- GET /api/v1/server/info → 200, info.database
- GET /api/v1/metrics → 200, metrics
- GET /api/v1/llm/providers, /llm/models, /llm/providers/ollama → 200
- GET /api/v1/llm/providers/<unknown> → 404 (unknown LLM provider)
- GET /api/v1/memory/systems, /memory/stats → 200
- POST /api/v1/auth/login empty body → 400 validation; bad creds → 401
- POST /api/v1/auth/logout (no header) → 400; POST /auth/refresh → 401
- GET on /tasks /projects /workers /sessions /users/me /system/stats /system/status /qa/sessions /screenshot/engines (no auth) → 401
- garbage bearer token → 401; unknown route → 404; OPTIONS /tasks → 204
- five sequential /health probes (load smoke) all 200

Honest skips

- **HXC-API-029 (JWT-mint/decode)** — /api/v1/auth/register returns 400 internal_auth_failed_create_user on this live DB, so no JWT can be minted in-test. Honest SKIP, not a fabricated PASS. This is a real, separate HelixCode defect (register broken) worth its own tracker item.
- **HXC-API-030 (WebSocket)** — the HelixQA http: action type drives HTTP only, and HelixCode's internal/server route table registers no WebSocket upgrade route. WebSocket coverage needs a dedicated ws: driver. Honest SKIP.

Evidence files

File	Contents
run_1.txt / run_2.txt / run_3.txt	Three consecutive full run transcripts
mutation-proof.txt	Anti-bluff: wrong expectation → FAIL exit 1
endpoint-probes.txt	Raw curl proof every asserted endpoint is real

File	Contents
runner/main.go + runner/go.mod	The standalone stdlib runner

Repeatability for the other banks

The pattern is repeatable for the other pure-server-API banks (entity-management, admin-operations, security-validation, performance-validation) **but with two structural caveats discovered here:**

1. Several banks (full-qa-api, and presumably the entity/admin ones) target the **Catalogizer** product, not HelixCode. They cannot be “rewritten in place” against HelixCode — they need either (a) running against a live Catalogizer catalog-api, or (b) a fresh HelixCode-targeted bank like this one. The cheap step-prose conversion alone is not sufficient when the whole API surface differs.
2. The canonical helix_qa/pkg/autonomous runner does not build in a partial checkout (missing owned submodules in its replace set). Either check out DocProcessor/LLMOrchestrator/LLMProvider/VisionEngine, or use a stdlib-only runner like this one. The bank JSON is portable across both.

For the GUI/CLI/WebSocket banks, the http:-only approach does not apply — they need the Playwright CDP runtime, shell: driving, or a ws: driver respectively (none of which exist wired today).

Sources verified

Internal source-of-truth files read during this work (no external web sources — this is a code/runtime audit, not service-documentation authoring):

- helix_code/internal/server/server.go (live route table)
- helix_code/internal/server/handlers.go + internal/auth/auth.go (login/register)
- helix_qa/pkg/testbank/schema.go (action + assertion schema)
- helix_qa/pkg/autonomous/http_executor.go (executor contract this runner mirrors)
- helix_qa/pkg/autonomous/bank_realbinary_test.go (existing HTTP-step test)
- live http://localhost:8080 probes (captured in endpoint-probes.txt)